

Module 3: Building APIs and Web Services

HTTP Basics

1. Overview of HTTP Protocol

What is HTTP?

HTTP (HyperText Transfer Protocol) is an application-layer protocol that defines how messages are formatted and transmitted between web clients and servers. It serves as the foundation of data communication on the World Wide Web. Understanding its structure, methods, and status codes is essential for web development and API design. Each HTTP method serves a specific purpose, from retrieving data with GET to creating resources with POST. Status codes provide standardized communication about request outcomes, enabling robust error handling and user feedback. Modern applications leverage HTTP's flexibility while adhering to RESTful principles to create scalable and maintainable systems.

Key Characteristics

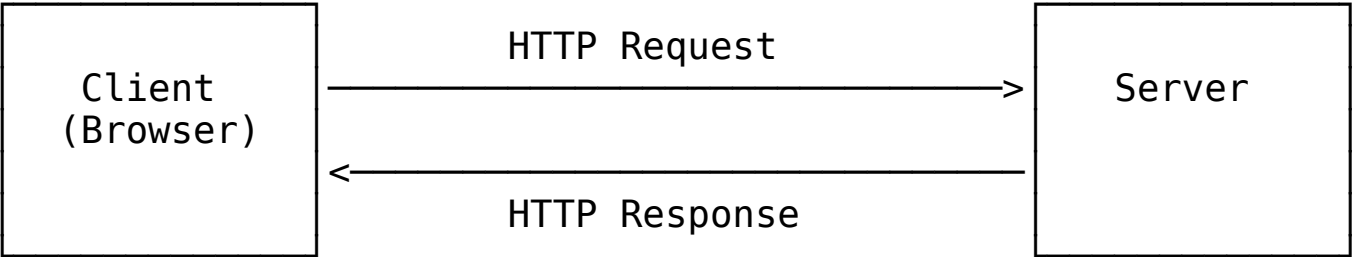
Stateless Protocol: Each request-response pair is independent. The server doesn't retain information about previous requests from the same client by default.

Client-Server Model: HTTP follows a request-response pattern where clients (browsers, mobile apps) initiate requests and servers provide responses.

Text-Based Protocol: HTTP messages are human-readable text, making debugging and analysis straightforward.

Port Usage: By default, HTTP uses port 80, while HTTPS (secure version) uses port 443.

HTTP Architecture



2. Standard Request and Response Structure

HTTP Request Structure

An HTTP request consists of four main components:

Request Line
Headers (multiple key-value pairs)
Blank Line
Body (optional) (message payload)

Request Line Format

METHOD /path/to/resource HTTP/version

Example Request:

```
POST /api/users HTTP/1.1
Host: example.com
Content-Type: application/json
Content-Length: 45
User-Agent: Mozilla/5.0
Accept: application/json
Authorization: Bearer token123

{"username":"john","email":"john@example.com"}
```

Request Components Breakdown

Request Line: Contains the HTTP method, resource path, and protocol version.

Headers: Metadata about the request providing additional context like content type, accepted formats, authentication credentials, and client information.

Blank Line: Separates headers from the body (mandatory even if there’s no body).

Body: Contains data being sent to the server (used in POST, PUT, PATCH requests).

HTTP Response Structure

Status Line
Headers (multiple key-value pairs)
Blank Line
Body (optional) (response payload)

Status Line Format

HTTP/version StatusCode ReasonPhrase

Example Response:

```
HTTP/1.1 200 OK
Date: Thu, 22 Jan 2026 10:30:00 GMT
Content-Type: application/json
Content-Length: 89
Server: Apache/2.4.41
Cache-Control: no-cache

{"id":123,"username":"john","email":"john@example.com","created":"2026-01-22T10:30:00Z"}
```

3. HTTP Methods: Structure and Usage

HTTP methods (also called verbs) indicate the desired action to be performed on a resource.

GET Method

Purpose: Retrieve data from the server without modifying it.

Request Body: Not used (data sent via URL query parameters).

Structure:

```
GET /api/users?page=1&limit=10 HTTP/1.1
Host: api.example.com
Accept: application/json
```

Response Example:

```
HTTP/1.1 200 OK
Content-Type: application/json

[{"id":1,"name":"Alice"}, {"id":2,"name":"Bob"}]
```

Use Cases: Fetching web pages, retrieving API data, searching, filtering results.

POST Method

Purpose: Submit data to create a new resource or trigger processing.

Request Body: Contains the data being sent.

Structure:

```
POST /api/users HTTP/1.1
Host: api.example.com
Content-Type: application/json
Content-Length: 58

{"name":"Charlie","email":"charlie@example.com","age":28}
```

Response Example:

```
HTTP/1.1 201 Created
Location: /api/users/3
Content-Type: application/json

{"id":3,"name":"Charlie","email":"charlie@example.com"}
```

Use Cases: Creating new resources, submitting forms, uploading files, triggering server-side operations.

PUT Method

Purpose: Update an existing resource or create it if it doesn't exist (full replacement).

Structure:: Similar to POST but targets a specific resource.

Use Cases: Complete resource updates, replacing entire documents.

PATCH Method

Purpose: Partially update an existing resource.

Request Body: Contains only the fields to be updated.

Structure: Similar to PUT but with partial data.

Use Cases: Updating specific fields without sending the entire resource.

DELETE Method

Purpose: Remove a resource from the server.

Request Body: Usually empty.

Structure:

```
DELETE /api/users/3 HTTP/1.1
Host: api.example.com
```

Response Example:

```
HTTP/1.1 204 No Content
```

Use Cases: Removing resources, canceling subscriptions, clearing data.

HEAD Method

Purpose: Same as GET but retrieves only headers, not the body.

Request Body: Not used.

Structure:

```
HEAD /api/users/3 HTTP/1.1
Host: api.example.com
```

Response Example:

```
HTTP/1.1 200 OK
Content-Type: application/json
```

Content-Length: 89
Last-Modified: Thu, 22 Jan 2026 10:30:00 GMT

(no body)

Use Cases: Checking if a resource exists, getting metadata, checking last modification time.

OPTIONS Method

Purpose: Describe communication options for the target resource.

Request Body: Not used.

Structure:

```
OPTIONS /api/users HTTP/1.1
Host: api.example.com
```

Response Example:

```
HTTP/1.1 200 OK
Allow: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Origin: *
```

Use Cases: CORS preflight requests, discovering allowed methods on a resource.

4. Important HTTP Status Codes

Status codes are three-digit numbers that indicate the result of an HTTP request. They are grouped into five categories.

Status Code Categories

Status Codes (3-digit)

- 1xx: Informational (request received, processing)
 - Rarely used in modern applications
- 2xx: Success (request successfully processed)
 - Action completed successfully
- 3xx: Redirection (further action needed)

- └─ Client must take additional action

- 4xx: Client Error (request contains errors)

- └─ Problem with the request itself

- 5xx: Server Error (server failed to fulfill request)

- └─ Server encountered an error

200 OK: Standard success response. Request succeeded.

201 Created: New resource created successfully (typically after POST).

301 Moved Permanently: Resource permanently moved to a new URL.

```
HTTP/1.1 301 Moved Permanently
```

```
Location: https://newsite.com/resource
```

400 Bad Request: Server cannot process the request due to client error (malformed syntax).

```
HTTP/1.1 400 Bad Request
```

```
Content-Type: application/json
```

```
{"error": "Invalid JSON format in request body"}
```

401 Unauthorized: Authentication required or failed.

```
HTTP/1.1 401 Unauthorized
```

```
WWW-Authenticate: Bearer realm="API"
```

```
{"error": "Authentication required"}
```

403 Forbidden: Server understood request but refuses to authorize it.

```
HTTP/1.1 403 Forbidden
```

```
{"error": "You don't have permission to access this resource"}
```

404 Not Found: Requested resource doesn't exist.

```
HTTP/1.1 404 Not Found
```

```
{"error": "User not found"}
```

405 Method Not Allowed: HTTP method not supported for this resource.

HTTP/1.1 405 Method Not Allowed

Allow: GET, POST

```
{"error": "DELETE method not allowed"}
```

500 Internal Server Error: Generic server error.

HTTP/1.1 500 Internal Server Error

```
{"error": "An unexpected error occurred"}
```

Status Code Quick Reference (For understanding purposes)

Code	Name	Category	Meaning
200	OK	Success	Request succeeded
201	Created	Success	New resource created
204	No Content	Success	Success but no content to return
301	Moved Permanently	Redirection	Resource moved permanently
302	Found	Redirection	Resource temporarily moved
304	Not Modified	Redirection	Resource unchanged (cache valid)
400	Bad Request	Client Error	Malformed request
401	Unauthorized	Client Error	Authentication required
403	Forbidden	Client Error	Access denied
404	Not Found	Client Error	Resource doesn't exist
409	Conflict	Client Error	Request conflicts with current state
422	Unprocessable Entity	Client Error	Validation failed
429	Too Many Requests	Client Error	Rate limit exceeded
500	Internal Server Error	Server Error	Generic server error
502	Bad Gateway	Server Error	Invalid upstream response
503	Service Unavailable	Server Error	Server temporarily unavailable

REST APIs Quick Overview

What is REST?

REST (Representational State Transfer) is an architectural style for building web services that use HTTP methods to perform operations on resources. RESTful APIs are designed to be stateless, scalable, and easy to maintain. They use standard HTTP methods (GET, POST, PUT, DELETE) to perform CRUD operations on resources identified by URIs. REST emphasizes a uniform interface and resource-based interactions, making it a popular choice for web API development.

Key Principles:

- **Resources:** Everything is a resource (User, Product, Order)
- **URIs:** Each resource has a unique identifier (URL)
- **HTTP Methods:** Use standard methods (GET, POST, PUT, DELETE)
- **Stateless:** Each request contains all necessary information
- **JSON/XML:** Data exchange format

REST vs Traditional Web Services

Traditional web services often use RPC-style endpoints that are action-based and may not follow RESTful principles. In contrast, RESTful APIs focus on resources and use standard HTTP methods to perform operations on those resources. This leads to more intuitive and scalable APIs. Using RESTful design allows for better separation of concerns, easier maintenance, and improved scalability compared to traditional web services. HTTP methods are used to indicate the desired action on a resource, and URIs are designed to represent resources rather than actions, making RESTful APIs more intuitive and easier to use. Common mappings of HTTP methods to CRUD operations in RESTful APIs are as follows:

Traditional:

POST /getUserById

POST /createUser

POST /updateUser

POST /deleteUser

RESTful:

GET /users/{id} - Get user

POST /users - Create user

PUT /users/{id} - Update user

DELETE /users/{id} - Delete user

OpenAPI

What is OpenAPI?

OpenAPI Specification (OAS) is a standard, language-agnostic interface description for HTTP APIs. It allows developers to define the structure of their APIs in a machine-readable format (YAML or JSON). This specification can be used to generate interactive documentation, client SDKs, server stubs, and perform automated testing. OpenAPI promotes consistency and standardization in API design, making it easier for developers to understand and consume APIs.

Key Benefits

- **Standardization:** Industry-standard format for API documentation
- **Auto-generation:** Generate client SDKs, server stubs, and documentation
- **Validation:** Validate requests and responses automatically
- **Testing:** Enable automated API testing
- **Discoverability:** Make APIs easier to understand and consume

Version Information

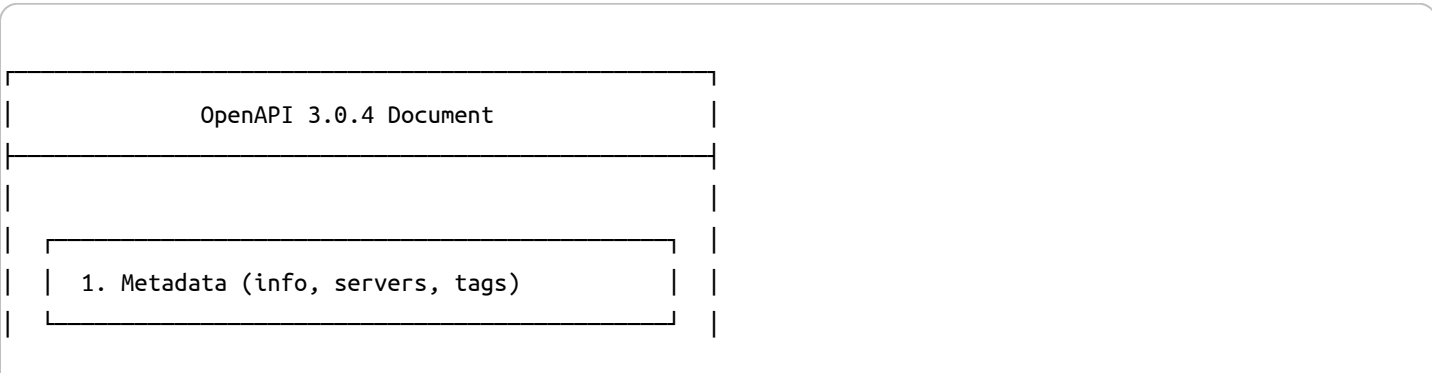
- **Current Version:** OpenAPI 3.0.4 (March 2024)
- **Format:** YAML or JSON
- **Previous Version:** OpenAPI 3.0.3, Swagger 2.0

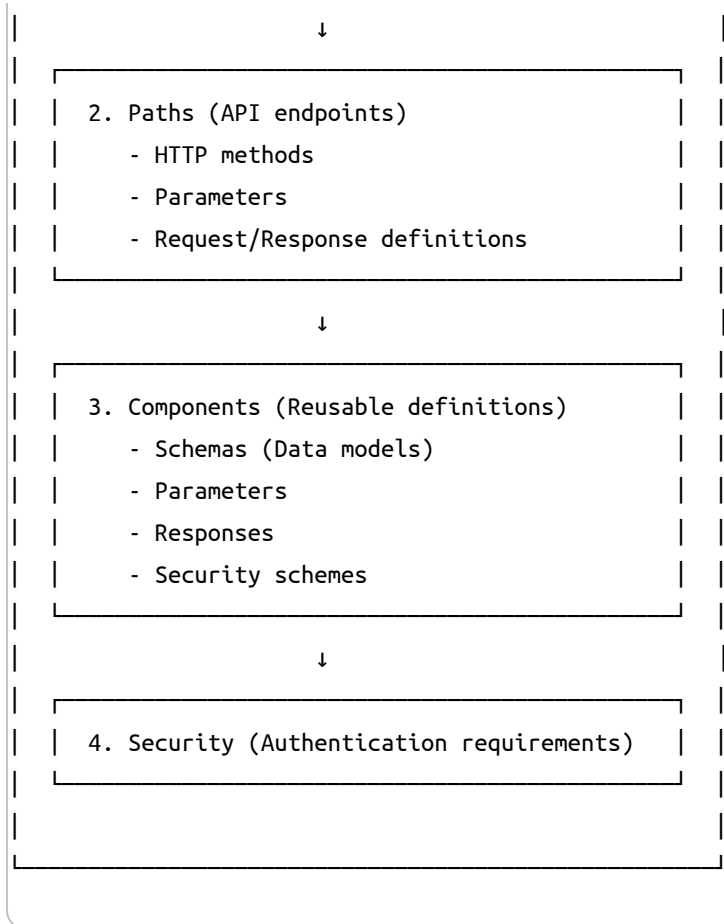
OpenAPI Specification Structure

High-Level Architecture

Different sections of the OpenAPI document include:

- **Info:** Contains metadata about the API (title, version, description).
- **Servers:** Lists the servers where the API is hosted.
- **Paths:** Defines the API endpoints, HTTP methods, parameters, request bodies, and responses.
- **Security:** Specifies the authentication methods required for the API.
- **Components:** Contains reusable definitions for schemas, parameters, responses, and security schemes.





Example OpenAPI Document

Below is an example of a simple OpenAPI 3.0.4 document in YAML format that defines a basic API for managing users.

```
openapi: 3.0.4
info:
  title: User Management API
  version: 1.0.0
  description: API for managing users in the system
servers:
  - url: https://api.example.com/v1
paths:
  /users:
    get:
      summary: Get a list of users
      responses:
        '200':
          description: A list of users
          content:
            application/json:
              schema:
                type: array
                items:
```

\$ref: '#/components/schemas/User'

post:

summary: Create a new user

requestBody:

required: true

content:

application/json:

schema:

\$ref: '#/components/schemas/User'

responses:

'201':

description: User created successfully

content:

application/json:

schema:

\$ref: '#/components/schemas/User'

/users/{id}:

get:

summary: Get a user by ID

parameters:

- name: id

in: path

required: true

schema:

type: integer

responses:

'200':

description: User details

content:

application/json:

schema:

\$ref: '#/components/schemas/User'

'404':

description: User not found

security:

- bearerAuth: []

components:

schemas:

User:

type: object

properties:

id:

type: integer

name:

type: string

email:

type: string

required:

- name

```
- email
securitySchemes:
  bearerAuth:
    type: http
    scheme: bearer
    bearerFormat: JWT
```

REST API using Spring Boot 3

What is @RestController?

`@RestController` is a specialized version of the `@Controller` annotation used to create RESTful web services in Spring Boot. RestControllers are designed to handle HTTP requests and return data (usually in JSON or XML format) rather than rendering views. When you annotate a class with `@RestController`, Spring automatically converts the return values of the methods into the appropriate format based on the client's request (using content negotiation). RestControllers are typically used to build APIs that serve data to clients, such as web applications, mobile apps, or other services. In spring boot `@RestController` is a convenient annotation that combines `@Controller` and `@ResponseBody`, eliminating the need to annotate each method with `@ResponseBody` to indicate that the return value should be serialized directly to the HTTP response body.

```
@RestController = @Controller + @ResponseBody
```

Key Characteristics:

- Automatically converts return values to JSON/XML
- Eliminates need for `@ResponseBody` on every method
- Designed for REST API development
- Returns data instead of views

Basic Usage

Simple RestController

```
@RestController
@RequestMapping("/api")
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello, World!";
    }
}
```

```
}
```

Above example defines a simple REST endpoint that returns a greeting message. The `@RequestMapping("/api")` sets the base path for all endpoints in this controller, and the `@GetMapping("/hello")` maps GET requests to the `hello()` method.

Access: GET `http://localhost:8080/api/hello`

Key Annotations

1. @RequestMapping

Maps HTTP requests to handler methods. Can be used at class and method level.

```
@RestController
@RequestMapping("/api/users") // Base path for all methods
public class UserController {

    @RequestMapping("/all") // /api/users/all
    public List<User> getAllUsers() {
        return userList;
    }
}
```

2. HTTP Method Annotations

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @GetMapping          // GET - Read
    @PostMapping         // POST - Create
    @PutMapping          // PUT - Update
    @PatchMapping        // PATCH - Partial Update
    @DeleteMapping       // DELETE - Delete
}
```

3. @PathVariable

Path variables for dynamic URL segments. These values are extracted from the URL and passed as method parameters. The name in `@PathVariable` should match the placeholder in the URL. Path variables are required by default, but you can make them optional by setting `required=false` and providing a default value.

```
// GET /api/users/123
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) {
    return findById(id);
}

// GET /api/users/123/orders/456
@GetMapping("/users/{userId}/orders/{orderId}")
public Order getOrder(
    @PathVariable Long userId,
    @PathVariable Long orderId) {
    return findOrder(userId, orderId);
}
```

4. @RequestParam

Query parameters are passed in the URL after the `?` and are used to filter or modify the request. They are optional by default, but you can make them required. Request parameters are typically used for filtering, pagination, or any additional data that doesn't fit well in the URL path. You can also provide default values for request parameters if they are not provided by the client.

```
// GET /api/search?keyword=java&page=1
@GetMapping("/search")
public List<Item> search(
    @RequestParam String keyword,
    @RequestParam(defaultValue = "0") int page) {
    return searchItems(keyword, page);
}

// Optional parameter
@GetMapping("/filter")
public List<Item> filter(
    @RequestParam(required = false) String category) {
    return filterItems(category);
}
```

5. @RequestBody

Request body is used to pass complex objects in the request payload, typically in JSON format. Spring automatically deserializes the JSON into the specified Java object. Request body is commonly used in POST and PUT requests to create or update resources. You can also use `@Valid` with `@RequestBody` to trigger validation of the incoming data based on constraints defined in the model class.

```
@PostMapping("/users")
public User createUser(@RequestBody User user) {
    return saveUser(user);
}

// Request JSON:
{
    "name": "John Doe",
    "email": "john@example.com"
}
```

6. @RequestHeader

RequestHeader is used to access HTTP headers sent by the client. You can specify the header name and Spring will inject its value into the method parameter. Request headers are often used for authentication tokens, content type, user agent information, and other metadata about the request. You can also set default values for headers if they are not provided by the client.

```
@GetMapping("/info")
public String getInfo(
    @RequestHeader("User-Agent") String userAgent) {
    return "Browser: " + userAgent;
}
```

Complete Example

```
@RestController
@RequestMapping("/api/books")
public class BookController {

    private List<Book> books = new ArrayList<>();
    private Long nextId = 1L;

    // GET all books
    @GetMapping
    public List<Book> getAllBooks() {
        return books;
    }

    // GET book by ID
    @GetMapping("/{id}")
    public Book getBook(@PathVariable Long id) {
```

```

        return books.stream()
            .filter(b -> b.getId().equals(id))
            .findFirst()
            .orElse(null);
    }

    // POST - Create book
    @PostMapping
    public Book createBook(@RequestBody Book book) {
        book.setId(nextId++);
        books.add(book);
        return book;
    }

    // PUT - Update book
    @PutMapping("/{id}")
    public Book updateBook(
        @PathVariable Long id,
        @RequestBody Book updatedBook) {

        for (int i = 0; i < books.size(); i++) {
            if (books.get(i).getId().equals(id)) {
                updatedBook.setId(id);
                books.set(i, updatedBook);
                return updatedBook;
            }
        }
        return null;
    }

    // DELETE book
    @DeleteMapping("/{id}")
    public String deleteBook(@PathVariable Long id) {
        books.removeIf(b -> b.getId().equals(id));
        return "Book deleted";
    }

    // Search with query parameter
    @GetMapping("/search")
    public List<Book> search(@RequestParam String title) {
        return books.stream()
            .filter(b -> b.getTitle().contains(title))
            .toList();
    }
}

// Book Model
class Book {
    private Long id;

```

```
private String title;
private String author;

// Constructors, Getters, Setters
}
```

Response Handling

1. Return Simple Types

```
@GetMapping("/message")
public String getMessage() {
    return "Hello"; // Returns: "Hello"
}
```

2. Return Objects (Auto-converted to JSON)

```
@GetMapping("/user")
public User getUser() {
    return new User("John", "john@example.com");
}

// Response:
{
  "name": "John",
  "email": "john@example.com"
}
```

3. Return Collections

```
@GetMapping("/users")
public List<User> getUsers() {
    return List.of(
        new User("John", "john@example.com"),
        new User("Jane", "jane@example.com")
    );
}

// Response: [ {...}, {...} ]
```

4. ResponseEntity for Status Control

```
@GetMapping("/user/{id}")
public ResponseEntity<User> getUser(@PathVariable Long id) {
    User user = findUser(id);

    if (user != null) {
        return ResponseEntity.ok(user); // 200 OK
    } else {
        return ResponseEntity.notFound().build(); // 404 Not Found
    }
}

@PostMapping("/users")
public ResponseEntity<User> createUser(@RequestBody User user) {
    User created = saveUser(user);
    return ResponseEntity
        .status(HttpStatus.CREATED) // 201 Created
        .body(created);
}
```

HTTP Status Codes

```
@RestController
public class StatusController {

    @GetMapping("/ok")
    public String ok() {
        return "Success"; // 200 OK (default)
    }

    @PostMapping("/create")
    @ResponseStatus(HttpStatus.CREATED) // 201
    public String create() {
        return "Created";
    }

    @DeleteMapping("/delete")
    @ResponseStatus(HttpStatus.NO_CONTENT) // 204
    public void delete() {
        // No content returned
    }
}
```

Common Status Codes:

- 200 OK - Success (default)
- 201 Created - Resource created
- 204 No Content - Success, no response body
- 400 Bad Request - Invalid request
- 404 Not Found - Resource not found
- 500 Internal Server Error - Server error

Exception Handling

Method-Level Exception Handler

Method level exception handlers allow you to handle exceptions specific to a controller. You can use the `@ExceptionHandler` annotation to define methods that will handle specific exceptions thrown by any method in the controller. This is useful for handling exceptions that are relevant only to that controller, allowing you to return custom error responses or status codes.

```
@RestController
public class ProductController {

    @GetMapping("/products/{id}")
    public Product getProduct(@PathVariable Long id) {
        if (id < 1) {
            throw new IllegalArgumentException("Invalid ID");
        }
        return findProduct(id);
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<String> handleBadRequest(
        IllegalArgumentException ex) {

        return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body(ex.getMessage());
    }
}
```

Global Exception Handler

Global exception handlers allow you to handle exceptions across the entire application in a centralized manner. By using the `@RestControllerAdvice` annotation, you can define a class that will intercept exceptions thrown by any controller and provide a consistent error response format. This is particularly useful for handling common exceptions like `ResourceNotFoundException`, validation errors, or any unhandled exceptions, ensuring that clients receive meaningful error messages and appropriate HTTP status codes.

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
        ResourceNotFoundException ex) {

        ErrorResponse error = new ErrorResponse(
            404,
            ex.getMessage(),
            System.currentTimeMillis()
        );

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(error);
    }
}

```

Data Validation

Add Validation Annotations

Validation annotations from the `jakarta.validation.constraints` package can be used to enforce constraints on request data. When combined with `@Valid`, Spring will automatically validate the incoming request body against these constraints and return a 400 Bad Request response if validation fails. In order to customize the error response, you can create a global exception handler that catches `MethodArgumentNotValidException` and formats the validation errors in a user-friendly way.

```

import jakarta.validation.constraints.*;

public class User {

    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 50)
    private String name;

    @Email(message = "Invalid email")
    @NotBlank
    private String email;

    @Min(18)
    @Max(100)
    private int age;
}

```

```
// Getters, Setters  
}
```

Use @Valid in Controller

@Valid triggers the validation process for the request body. If any validation constraints are violated, Spring will throw a **MethodArgumentNotValidException**, which can be handled to return detailed error messages to the client.

```
@RestController  
@RequestMapping("/api/users")  
public class UserController {  
  
    @PostMapping  
    public ResponseEntity<User> createUser(  
        @Valid @RequestBody User user) {  
  
        // If validation fails, returns 400 Bad Request  
        User created = saveUser(user);  
        return ResponseEntity.status(HttpStatus.CREATED)  
            .body(created);  
    }  
}
```

Handle Validation Errors

Handle validation errors globally to provide consistent error responses. The example below captures all validation errors and returns a structured response containing the field names and corresponding error messages. **MethodArgumentNotValidException** is thrown when validation on an argument annotated with @Valid fails. By catching this exception in a global exception handler, you can extract the validation errors and return them in a user-friendly format, such as a JSON object containing the field names and error messages.

```
@RestControllerAdvice  
public class ValidationExceptionHandler {  
  
    @ExceptionHandler(MethodArgumentNotValidException.class)  
    public ResponseEntity<Map<String, String>> handleValidation(  
        MethodArgumentNotValidException ex) {  
  
        Map<String, String> errors = new HashMap<>();  
  
        ex.getBindingResult().getAllErrors().forEach(error -> {
```

```

        String field = ((FieldError) error).getField();
        String message = error.getDefaultMessage();
        errors.put(field, message);
    });

    return ResponseEntity
        .status(HttpStatus.BAD_REQUEST)
        .body(errors);
}
}

```

Common Validation Annotations

```

@NotNull           // Cannot be null
@NotEmpty          // Cannot be null or empty (String, Collection)
@NotBlank         // Cannot be null, empty, or whitespace (String only)
@Size(min, max)   // Size constraints (String, Collection, Array)
@Min(value)       // Minimum numeric value
@Max(value)       // Maximum numeric value
>Email           // Valid email format
@Pattern(regex)   // Matches regex pattern
@Past            // Date in the past
@Future          // Date in the future
@Positive        // Positive number
@Negative        // Negative number

```

Best Practices

1. Use Proper HTTP Methods

Use the correct HTTP method for each operation to follow RESTful principles. This makes your API more intuitive and easier to use. Don't use POST for actions that are meant to read data, and avoid using GET for operations that modify data.

```

// GOOD
@GetMapping("/users")           // Read
@PostMapping("/users")         // Create
@PutMapping("/users/{id}")      // Update
@DeleteMapping("/users/{id}")   // Delete

// BAD
@PostMapping("/getUsers")
@PostMapping("/createUser")

```

2. Use Meaningful URIs

Resource-based URIs are more intuitive and easier to understand than action-based URIs. Use nouns to represent resources and avoid verbs in the URI path. Don't include implementation details or actions in the URI. Instead, use HTTP methods to indicate the action being performed on the resource.

```
// GOOD - Resource-based
/api/customers
/api/customers/{id}
/api/customers/{id}/orders

// BAD - Action-based
/api/getCustomers
/api/createCustomer
```

3. Return Appropriate Status Codes

Returning the correct HTTP status codes helps clients understand the result of their requests and handle responses appropriately. Use 200 OK for successful GET requests, 201 Created for successful POST requests that create resources, and 204 No Content for successful DELETE requests. Avoid returning 200 OK for all responses, as it can be misleading and does not provide enough information about the outcome of the request.

```
@PostMapping("/users")
public ResponseEntity<User> create(@RequestBody User user) {
    return ResponseEntity
        .status(HttpStatus.CREATED) // 201 instead of 200
        .body(user);
}

@DeleteMapping("/users/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id) {
    deleteUser(id);
    return ResponseEntity.noContent().build(); // 204
}
```

4. Use Constructor Injection

Constructor injection is generally recommended over field injection because it promotes immutability, makes dependencies explicit, and is easier to test. With constructor injection, you can easily see what dependencies a class requires by looking at its constructor. It also allows you to create immutable objects, which can help prevent bugs and improve thread safety. Field

injection, on the other hand, can lead to hidden dependencies and makes it harder to write unit tests for the class.

```
// GOOD
@RestController
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }
}

// AVOID
@RestController
public class UserController {
    @Autowired
    private UserService userService; // Field injection
}
```

5. Version Your API

API versioning allows you to make changes to your API without breaking existing clients. You can version your API using the URL, request header, or query parameter. URL versioning is the most common and straightforward approach, where you include the version number in the URI path. This makes it clear to clients which version of the API they are using and allows you to maintain multiple versions simultaneously.

```
@RestController
@RequestMapping("/api/v1/products") // Version in URL
public class ProductController {
    // ...
}
```

Note: Avoid using query parameters for versioning, as it can be less intuitive and may lead to confusion for clients. Header-based versioning can be more flexible but requires clients to set the appropriate headers, which may not be as straightforward as URL versioning. Hence URL versioning is often preferred for its simplicity and clarity.

Quick Reference (For understanding purposes)

Annotation Summary

Annotation	Purpose
@RestController	Defines REST controller
@RequestMapping	Maps requests to handler
@GetMapping	Maps GET requests
@PostMapping	Maps POST requests
@PutMapping	Maps PUT requests
@DeleteMapping	Maps DELETE requests
@PathVariable	Extracts URI variable
@RequestParam	Extracts query parameter
@RequestBody	Binds request body to object
@RequestHeader	Extracts HTTP header
@Valid	Enables validation
@ResponseStatus	Sets HTTP status code

Common Patterns

```
// Basic CRUD endpoints
GET    /api/resources           // List all
GET    /api/resources/{id}      // Get one
POST   /api/resources           // Create
PUT    /api/resources/{id}      // Update
DELETE /api/resources/{id}      // Delete

// Nested resources
GET    /api/users/{id}/orders
POST   /api/users/{id}/orders

// Search/Filter
GET    /api/products?category=books&sort=price
```

Testing with cURL

cURL is a command-line tool for making HTTP requests. You can use it to test your REST API endpoints by sending various types of requests (GET, POST, PUT, DELETE) and inspecting the responses. Below are examples of how to use cURL to interact with a REST API built with Spring Boot.

GET request

```
curl http://localhost:8080/api/books
```

GET with path variable

```
curl http://localhost:8080/api/books/1
```

POST request

```
curl -X POST http://localhost:8080/api/books \  
-H "Content-Type: application/json" \  
-d '{"title":"Spring Boot","author":"John Doe"}'
```

PUT request

```
curl -X PUT http://localhost:8080/api/books/1 \  
-H "Content-Type: application/json" \  
-d '{"title":"Updated Title","author":"Jane Doe"}'
```

DELETE request

```
curl -X DELETE http://localhost:8080/api/books/1
```